RISC-V 5-Stage Pipeline Architecture

Table of Contents

1 RISC-V 5-Stage Pipeline Architecture **2**

1.1 Project Introduction 2
1.2 Fetch Cycle 2
1.3 Decode Cycle 4
1.4 Execute Cycle 8
1.5 Memory Cycle 11
1.6 Writeback Cycle 13
1.7 Pipeline Top 13
1.8 Hazard Unit 17
1.9 Pipeline Testbench 18
1.10 Instruction Sequence and Pipeline Operation 19

2 Simulation Waveform **19**

3 Conclusion **21**

1 RISC-V 5-Stage Pipeline Architecture

1.1 Project Introduction

This project implements a 32-bit RISC-V processor using a five-stage pipelined architecture in Verilog. The processor divides instruction execution into five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Pipeline registers are used between consecutive stages to allow multiple instructions to be processed simultaneously. The design also includes a hazard unit with data forwarding to handle data dependencies between instructions. The processor was simulated using Icarus Verilog and a testbench was used to verify the operation of the complete pipeline.

The instruction memory is initialized using a hexadecimal memory file containing RISC-V instructions. The instructions used for testing are shown in Table 1.

Table 1: Instruction sequence used for testing

PC	Machine Code	Instruction
0x00	0x00500293	ADDI x5, x0, 5
0x04	0x00300313	ADDI x6, x0, 3
0x08	0x006283B3	ADD x7, x5, x6
0x0C	0x00002403	LW x8, 0(x0)
0x10	0x00100493	ADDI x9, x0, 1
0x14	0x00940533	ADD x10, x8, x9

The instruction sequence contains arithmetic, immediate, and load instructions. It also creates data dependencies, such as the use of registers x5 and x6 by the ADD instruction and the use of x8 and x9 by the final ADD instruction. These dependencies are useful for verifying the forwarding mechanism of the pipeline.

1.2 Fetch Cycle

The Fetch Cycle is implemented in the `Fetch_Cycle.v` module. Its main purpose is to obtain the instruction from instruction memory using the current Program Counter (PC). The PC normally increases by 4 bytes for every

instruction because each instruction is 32 bits wide. A multiplexer selects either the normal next PC, `PC+4`, or a branch target address depending on the `PCSrcE` signal. The fetched instruction, current PC, and PC+4 are stored in the IF/ID pipeline registers at the rising edge of the clock. These registered values are then supplied to the Decode stage.

```
module fetch_cycle(clk, rst, PCSrcE, PCTargetE, InstrD, PCD,
   PCPlus4D);

input clk, rst, PCSrcE;
input [31:0] PCTargetE;
output [31:0] InstrD, PCD, PCPlus4D;

wire [31:0] PC_F, PCF, PCPlus4F, InstrF;
reg [31:0] PCF_reg, PCPlus4F_reg, InstrF_reg;

    Mux PC_Mux(
                .a(PCPlus4F),
                .b(PCTargetE),
                .s(PCSrcE),
                .y(PC_F)
                 );

    PC Program_Counter(
                 .clk(clk),
                 .rst(rst),
                 .PC(PCF),
                 .PC_Next(PC_F)
                 );

    // Declare Instruction Memory
    Instruction_Memory IMEM(
                 .rst(rst),
                 .A(PCF),
                 .RD(InstrF)
                 );

    // Declare PC adder
    Adder PC_adder(
                 .a(PCF),
```

```
                .b(32'h00000004),
                .c(PCPlus4F)
                );

    // Fetch Cycle Register Logic
    always @(posedge clk or negedge rst) begin
        if(rst == 1'b0) begin
            InstrF_reg <= 32'h00000000;
            PCF_reg <= 32'h00000000;
            PCPlus4F_reg <= 32'h00000000;
        end
        else begin
            InstrF_reg <= InstrF;
            PCF_reg <= PCF;
            PCPlus4F_reg <= PCPlus4F;
        end
    end

    // Assigning Registers Value to the Output port
    assign InstrD = (rst == 1'b0) ? 32'h00000000 : InstrF_reg;
    assign PCD = (rst == 1'b0) ? 32'h00000000 : PCF_reg;
    assign PCPlus4D = (rst == 1'b0) ? 32'h00000000 : PCPlus4F_reg;

endmodule
```

1.3 Decode Cycle

The Decode Cycle is implemented in the `Decode_Cycle.v` module. In this stage, the fetched instruction is decoded to generate the required control signals. The register file provides the values of the two source registers specified by the instruction, while the `Sign_Extend` module generates the required immediate value. The control unit determines signals such as `RegWrite`, `ALUSrc`, `MemWrite`, `ResultSrc`, and `Branch`. The decoded data and control signals are stored in the ID/EX pipeline registers before being passed to the Execute stage.

```
module decode_cycle(clk, rst, InstrD, PCD, PCPlus4D, RegWriteW, RDW
    , ResultW, RegWriteE, ALUSrcE, MemWriteE, ResultSrcE,
    BranchE,  ALUControlE, RD1_E, RD2_E, Imm_Ext_E, RD_E, PCE,
```

```
    PCPlus4E, RS1_E, RS2_E);

    // Declaring I/O
    input clk, rst, RegWriteW;
    input [4:0] RDW;
    input [31:0] InstrD, PCD, PCPlus4D, ResultW;

    output RegWriteE,ALUSrcE,MemWriteE,ResultSrcE,BranchE;
    output [2:0] ALUControlE;
    output [31:0] RD1_E, RD2_E, Imm_Ext_E;
    output [4:0] RS1_E, RS2_E, RD_E;
    output [31:0] PCE, PCPlus4E;

    // Declare Interim Wires
    wire RegWriteD,ALUSrcD,MemWriteD,ResultSrcD,BranchD;
    wire [1:0] ImmSrcD;
    wire [2:0] ALUControlD;
    wire [31:0] RD1_D, RD2_D, Imm_Ext_D;

    // Declaration of Interim Register
    reg RegWriteD_r,ALUSrcD_r,MemWriteD_r,ResultSrcD_r,BranchD_r;
    reg [2:0] ALUControlD_r;
    reg [31:0] RD1_D_r, RD2_D_r, Imm_Ext_D_r;
    reg [4:0] RD_D_r, RS1_D_r, RS2_D_r;
    reg [31:0] PCD_r, PCPlus4D_r;


    // Initiate the modules
    // Control Unit
    Control_Unit_Top control (
                            .Op(InstrD[6:0]),
                            .RegWrite(RegWriteD),
                            .ImmSrc(ImmSrcD),
                            .ALUSrc(ALUSrcD),
                            .MemWrite(MemWriteD),
                            .ResultSrc(ResultSrcD),
                            .Branch(BranchD),
                            .funct3(InstrD[14:12]),
                            .funct7(InstrD[31:25]),
                            .ALUControl(ALUControlD)
```

```
                            );

    // Register File
    Register_File rf (
                        .clk(clk),
                        .rst(rst),
                        .WE3(RegWriteW),
                        .WD3(ResultW),
                        .A1(InstrD[19:15]),
                        .A2(InstrD[24:20]),
                        .A3(RDW),
                        .RD1(RD1_D),
                        .RD2(RD2_D)
                        );

    // Sign Extension
    Sign_Extend extension (
                        .In(InstrD[31:0]),
                        .Imm_Ext(Imm_Ext_D),
                        .ImmSrc(ImmSrcD)
                        );

    // Declaring Register Logic
    always @(posedge clk or negedge rst) begin
        if(rst == 1'b0) begin
            RegWriteD_r <= 1'b0;
            ALUSrcD_r <= 1'b0;
            MemWriteD_r <= 1'b0;
            ResultSrcD_r <= 1'b0;
            BranchD_r <= 1'b0;
            ALUControlD_r <= 3'b000;
            RD1_D_r <= 32'h00000000;
            RD2_D_r <= 32'h00000000;
            Imm_Ext_D_r <= 32'h00000000;
            RD_D_r <= 5'h00;
            PCD_r <= 32'h00000000;
            PCPlus4D_r <= 32'h00000000;
            RS1_D_r <= 5'h00;
            RS2_D_r <= 5'h00;
        end
```

```
        else begin
            RegWriteD_r <= RegWriteD;
            ALUSrcD_r <= ALUSrcD;
            MemWriteD_r <= MemWriteD;
            ResultSrcD_r <= ResultSrcD;
            BranchD_r <= BranchD;
            ALUControlD_r <= ALUControlD;
            RD1_D_r <= RD1_D;
            RD2_D_r <= RD2_D;
            Imm_Ext_D_r <= Imm_Ext_D;
            RD_D_r <= InstrD[11:7];
            PCD_r <= PCD;
            PCPlus4D_r <= PCPlus4D;
            RS1_D_r <= InstrD[19:15];
            RS2_D_r <= InstrD[24:20];
        end
    end

    // Output asssign statements
    assign RegWriteE = RegWriteD_r;
    assign ALUSrcE = ALUSrcD_r;
    assign MemWriteE = MemWriteD_r;
    assign ResultSrcE = ResultSrcD_r;
    assign BranchE = BranchD_r;
    assign ALUControlE = ALUControlD_r;
    assign RD1_E = RD1_D_r;
    assign RD2_E = RD2_D_r;
    assign Imm_Ext_E = Imm_Ext_D_r;
    assign RD_E = RD_D_r;
    assign PCE = PCD_r;
    assign PCPlus4E = PCPlus4D_r;
    assign RS1_E = RS1_D_r;
    assign RS2_E = RS2_D_r;

endmodule
```

1.4 Execute Cycle

The Execute Cycle is implemented in the `Execute_Cycle.v` module. This stage performs arithmetic and logical operations using the ALU. Two 3-to-1 multiplexers select the operands for the ALU and allow either register values or forwarded values from later pipeline stages to be used. A separate multiplexer selects either the second register operand or the sign-extended immediate depending on the `ALUSrcE` signal. The ALU performs operations such as addition, subtraction, AND, OR, and comparison. For branch instructions, an adder calculates the branch target address and the ALU zero output is used to generate the branch control signal. The ALU result and required control information are then stored in the EX/MEM pipeline registers.

```
1  module execute_cycle(clk, rst, RegWriteE, ALUSrcE, MemWriteE,
       ResultSrcE, BranchE, ALUControlE,
2      RD1_E, RD2_E, Imm_Ext_E, RD_E, PCE, PCPlus4E, PCSrcE, PCTargetE
       , RegWriteM, MemWriteM, ResultSrcM, RD_M, PCPlus4M, WriteDataM,
       ALU_ResultM, ResultW, ForwardA_E, ForwardB_E);
3
4      // Declaration I/Os
5      input clk, rst, RegWriteE,ALUSrcE,MemWriteE,ResultSrcE,BranchE;
6      input [2:0] ALUControlE;
7      input [31:0] RD1_E, RD2_E, Imm_Ext_E;
8      input [4:0] RD_E;
9      input [31:0] PCE, PCPlus4E;
10     input [31:0] ResultW;
11     input [1:0] ForwardA_E, ForwardB_E;
12
13     output PCSrcE, RegWriteM, MemWriteM, ResultSrcM;
14     output [4:0] RD_M;
15     output [31:0] PCPlus4M, WriteDataM, ALU_ResultM;
16     output [31:0] PCTargetE;
17
18     // Declaration of Interim Wires
19     wire [31:0] Src_A, Src_B_interim, Src_B;
20     wire [31:0] ResultE;
21     wire ZeroE;
22
23     // Declaration of Register
```

```
    reg RegWriteE_r, MemWriteE_r, ResultSrcE_r;
    reg [4:0] RD_E_r;
    reg [31:0] PCPlus4E_r, RD2_E_r, ResultE_r;

    // Declaration of Modules
    // 3 by 1 Mux for Source A
    Mux_3_X_1 srca_mux (
                        .a(RD1_E),
                        .b(ResultW),
                        .c(ALU_ResultM),
                        .s(ForwardA_E),
                        .y(Src_A)
                        );

    // 3 by 1 Mux for Source B
    Mux_3_X_1 srcb_mux (
                        .a(RD2_E),
                        .b(ResultW),
                        .c(ALU_ResultM),
                        .s(ForwardB_E),
                        .y(Src_B_interim)
                        );
    // ALU Src Mux
    Mux alu_src_mux (
            .a(Src_B_interim),
            .b(Imm_Ext_E),
            .s(ALUSrcE),
            .y(Src_B)
            );

    // ALU Unit
    ALU alu (
            .A(Src_A),
            .B(Src_B),
            .Result(ResultE),
            .ALUControl(ALUControlE),
            .OverFlow(),
            .Carry(),
            .Zero(ZeroE),
            .Negative()
```

```
64              );
65
66      // Adder
67      Adder branch_adder (
68              .a(PCE),
69              .b(Imm_Ext_E),
70              .c(PCTargetE)
71              );
72
73      // Register Logic
74      always @(posedge clk or negedge rst) begin
75          if(rst == 1'b0) begin
76              RegWriteE_r <= 1'b0;
77              MemWriteE_r <= 1'b0;
78              ResultSrcE_r <= 1'b0;
79              RD_E_r <= 5'h00;
80              PCPlus4E_r <= 32'h00000000;
81              RD2_E_r <= 32'h00000000;
82              ResultE_r <= 32'h00000000;
83          end
84          else begin
85              RegWriteE_r <= RegWriteE;
86              MemWriteE_r <= MemWriteE;
87              ResultSrcE_r <= ResultSrcE;
88              RD_E_r <= RD_E;
89              PCPlus4E_r <= PCPlus4E;
90              RD2_E_r <= Src_B_interim;
91              ResultE_r <= ResultE;
92          end
93      end
94
95      // Output Assignments
96      assign PCSrcE = ZeroE &  BranchE;
97      assign RegWriteM = RegWriteE_r;
98      assign MemWriteM = MemWriteE_r;
99      assign ResultSrcM = ResultSrcE_r;
100     assign RD_M = RD_E_r;
101     assign PCPlus4M = PCPlus4E_r;
102     assign WriteDataM = RD2_E_r;
103     assign ALU_ResultM = ResultE_r;
```

```
104
105 endmodule
```

1.5 Memory Cycle

The Memory Cycle is implemented in the `Memory_Cycle.v` module. This stage interacts with the data memory for load and store operations. The ALU result generated in the Execute stage is used as the memory address. For a store instruction, the value from the source register is written into memory when `MemWriteM` is asserted. For a load instruction, data is read from the corresponding memory location. The memory output, ALU result, destination register, and control signals are stored in the MEM/WB pipeline registers and passed to the Write Back stage.

```
1  module memory_cycle(clk, rst, RegWriteM, MemWriteM, ResultSrcM,
       RD_M, PCPlus4M, WriteDataM,
2      ALU_ResultM, RegWriteW, ResultSrcW, RD_W, PCPlus4W, ALU_ResultW
       , ReadDataW);
3
4      // Declaration of I/Os
5      input clk, rst, RegWriteM, MemWriteM, ResultSrcM;
6      input [4:0] RD_M;
7      input [31:0] PCPlus4M, WriteDataM, ALU_ResultM;
8
9      output RegWriteW, ResultSrcW;
10     output [4:0] RD_W;
11     output [31:0] PCPlus4W, ALU_ResultW, ReadDataW;
12
13     // Declaration of Interim Wires
14     wire [31:0] ReadDataM;
15
16     // Declaration of Interim Registers
17     reg RegWriteM_r, ResultSrcM_r;
18     reg [4:0] RD_M_r;
19     reg [31:0] PCPlus4M_r, ALU_ResultM_r, ReadDataM_r;
20
21     // Declaration of Module Initiation
22     Data_Memory dmem (
23                          .clk(clk),
```

```
                        .rst(rst),
                        .WE(MemWriteM),
                        .WD(WriteDataM),
                        .A(ALU_ResultM),
                        .RD(ReadDataM)
                    );

    // Memory Stage Register Logic
    always @(posedge clk or negedge rst) begin
        if (rst == 1'b0) begin
            RegWriteM_r <= 1'b0;
            ResultSrcM_r <= 1'b0;
            RD_M_r <= 5'h00;
            PCPlus4M_r <= 32'h00000000;
            ALU_ResultM_r <= 32'h00000000;
            ReadDataM_r <= 32'h00000000;
        end
        else begin
            RegWriteM_r <= RegWriteM;
            ResultSrcM_r <= ResultSrcM;
            RD_M_r <= RD_M;
            PCPlus4M_r <= PCPlus4M;
            ALU_ResultM_r <= ALU_ResultM;
            ReadDataM_r <= ReadDataM;
        end
    end

    // Declaration of output assignments
    assign RegWriteW = RegWriteM_r;
    assign ResultSrcW = ResultSrcM_r;
    assign RD_W = RD_M_r;
    assign PCPlus4W = PCPlus4M_r;
    assign ALU_ResultW = ALU_ResultM_r;
    assign ReadDataW = ReadDataM_r;

endmodule
```

1.6 Writeback Cycle

The Writeback Cycle is implemented in the `Writeback_Cycle.v` module. This is the final stage of the pipeline and selects the value that must be written back to the register file. A multiplexer selects between the ALU result and the data obtained from memory. For arithmetic instructions, the ALU result is selected, while for load instructions, the memory data is selected. The resulting value, represented by `ResultW`, is connected back to the Decode stage and is written into the destination register when `RegWriteW` is enabled.

```
module writeback_cycle(clk, rst, ResultSrcW, PCPlus4W, ALU_ResultW,
    ReadDataW, ResultW);

// Declaration of IOs
input clk, rst, ResultSrcW;
input [31:0] PCPlus4W, ALU_ResultW, ReadDataW;

output [31:0] ResultW;

// Declaration of Module
Mux result_mux (
                .a(ALU_ResultW),
                .b(ReadDataW),
                .s(ResultSrcW),
                .y(ResultW)
                );
endmodule
```

1.7 Pipeline Top

The complete processor is connected in the `Pipeline_Top.v` module. It integrates the five pipeline stages along with the hazard unit, register file, ALU, memories, control unit, multiplexers, and other supporting modules. The output of one stage is connected to the input of the following stage through pipeline registers. The Fetch, Decode, Execute, Memory, and Writeback stages therefore operate concurrently on different instructions. The Pipeline Top module also connects the forwarding signals generated by the Hazard Unit to the Execute stage, allowing dependent instructions to use recently

generated results without waiting for normal register writeback.

```
module Pipeline_Top(clk, rst);

    // Declaration of I/O
    input clk, rst;

    // Declaration of Interim Wires
    wire PCSrcE, RegWriteW, RegWriteE, ALUSrcE, MemWriteE,
    ResultSrcE, BranchE, RegWriteM, MemWriteM, ResultSrcM,
    ResultSrcW;
    wire [2:0] ALUControlE;
    wire [4:0] RD_E, RD_M, RDW;
    wire [31:0] PCTargetE, InstrD, PCD, PCPlus4D, ResultW, RD1_E,
    RD2_E, Imm_Ext_E, PCE, PCPlus4E, PCPlus4M, WriteDataM,
    ALU_ResultM;
    wire [31:0] PCPlus4W, ALU_ResultW, ReadDataW;
    wire [4:0] RS1_E, RS2_E;
    wire [1:0] ForwardBE, ForwardAE;


    // Module Initiation
    // Fetch Stage
    fetch_cycle Fetch (
                        .clk(clk),
                        .rst(rst),
                        .PCSrcE(PCSrcE),
                        .PCTargetE(PCTargetE),
                        .InstrD(InstrD),
                        .PCD(PCD),
                        .PCPlus4D(PCPlus4D)
                    );

    // Decode Stage
    decode_cycle Decode (
                        .clk(clk),
                        .rst(rst),
                        .InstrD(InstrD),
                        .PCD(PCD),
                        .PCPlus4D(PCPlus4D),
                        .RegWriteW(RegWriteW),
```

```
                        .RDW(RDW),
                        .ResultW(ResultW),
                        .RegWriteE(RegWriteE),
                        .ALUSrcE(ALUSrcE),
                        .MemWriteE(MemWriteE),
                        .ResultSrcE(ResultSrcE),
                        .BranchE(BranchE),
                        .ALUControlE(ALUControlE),
                        .RD1_E(RD1_E),
                        .RD2_E(RD2_E),
                        .Imm_Ext_E(Imm_Ext_E),
                        .RD_E(RD_E),
                        .PCE(PCE),
                        .PCPlus4E(PCPlus4E),
                        .RS1_E(RS1_E),
                        .RS2_E(RS2_E)
                    );

    // Execute Stage
    execute_cycle Execute (
                        .clk(clk),
                        .rst(rst),
                        .RegWriteE(RegWriteE),
                        .ALUSrcE(ALUSrcE),
                        .MemWriteE(MemWriteE),
                        .ResultSrcE(ResultSrcE),
                        .BranchE(BranchE),
                        .ALUControlE(ALUControlE),
                        .RD1_E(RD1_E),
                        .RD2_E(RD2_E),
                        .Imm_Ext_E(Imm_Ext_E),
                        .RD_E(RD_E),
                        .PCE(PCE),
                        .PCPlus4E(PCPlus4E),
                        .PCSrcE(PCSrcE),
                        .PCTargetE(PCTargetE),
                        .RegWriteM(RegWriteM),
                        .MemWriteM(MemWriteM),
                        .ResultSrcM(ResultSrcM),
                        .RD_M(RD_M),
```

```
76                          .PCPlus4M(PCPlus4M),
77                          .WriteDataM(WriteDataM),
78                          .ALU_ResultM(ALU_ResultM),
79                          .ResultW(ResultW),
80                          .ForwardA_E(ForwardAE),
81                          .ForwardB_E(ForwardBE)
82                      );
83
84      // Memory Stage
85      memory_cycle Memory (
86                          .clk(clk),
87                          .rst(rst),
88                          .RegWriteM(RegWriteM),
89                          .MemWriteM(MemWriteM),
90                          .ResultSrcM(ResultSrcM),
91                          .RD_M(RD_M),
92                          .PCPlus4M(PCPlus4M),
93                          .WriteDataM(WriteDataM),
94                          .ALU_ResultM(ALU_ResultM),
95                          .RegWriteW(RegWriteW),
96                          .ResultSrcW(ResultSrcW),
97                          .RD_W(RDW),
98                          .PCPlus4W(PCPlus4W),
99                          .ALU_ResultW(ALU_ResultW),
100                         .ReadDataW(ReadDataW)
101                     );
102
103     // Write Back Stage
104     writeback_cycle WriteBack (
105                         .clk(clk),
106                         .rst(rst),
107                         .ResultSrcW(ResultSrcW),
108                         .PCPlus4W(PCPlus4W),
109                         .ALU_ResultW(ALU_ResultW),
110                         .ReadDataW(ReadDataW),
111                         .ResultW(ResultW)
112                     );
113
114     // Hazard Unit
115     hazard Forwarding_block (
```

```
                        .rst(rst),
                        .RegWriteM(RegWriteM),
                        .RegWriteW(RegWriteW),
                        .RD_M(RD_M),
                        .RD_W(RDW),
                        .Rs1_E(RS1_E),
                        .Rs2_E(RS2_E),
                        .ForwardAE(ForwardAE),
                        .ForwardBE(ForwardBE)
                        );
endmodule
```

1.8 Hazard Unit

The Hazard Unit is implemented in the `Hazard.v` module and is mainly used to handle data hazards through forwarding. A data hazard occurs when an instruction requires a register value that is being produced by an earlier instruction still present in the pipeline. The Hazard Unit compares the source registers of the instruction in the Execute stage with the destination registers in the Memory and Writeback stages. If a match is detected and the corresponding instruction writes to a register, the unit generates the `ForwardAE` and `ForwardBE` control signals. These signals control the 3-to-1 multiplexers in the Execute stage so that the most recent result can be directly forwarded from the Memory or Writeback stage. Register x0 is excluded from forwarding because its value must always remain zero.

```
module hazard(rst, RegWriteM, RegWriteW, RD_M, RD_W, Rs1_E, Rs2_E,
    ForwardAE, ForwardBE);

    // Declaration of I/Os
    input rst, RegWriteM, RegWriteW;
    input [4:0] RD_M, RD_W, Rs1_E, Rs2_E;
    output [1:0] ForwardAE, ForwardBE;

    assign ForwardAE = (rst == 1'b0) ? 2'b00 :
                       ((RegWriteM == 1'b1) & (RD_M != 5'h00) & (
    RD_M == Rs1_E)) ? 2'b10 :
                       ((RegWriteW == 1'b1) & (RD_W != 5'h00) & (
    RD_W == Rs1_E)) ? 2'b01 : 2'b00;
```

```
11
12      assign ForwardBE = (rst == 1'b0) ? 2'b00 :
13                         ((RegWriteM == 1'b1) & (RD_M != 5'h00) & (
    RD_M == Rs2_E)) ? 2'b10 :
14                         ((RegWriteW == 1'b1) & (RD_W != 5'h00) & (
    RD_W == Rs2_E)) ? 2'b01 : 2'b00;
15
16  endmodule
```

1.9 Pipeline Testbench

The complete pipeline is verified using the `pipeline_tb.v` testbench. The testbench generates the clock signal with a 100 ns clock period and initially keeps the processor in reset. After 200 ns, reset is released and the pipeline is allowed to execute the instructions for a further 1000 ns. The testbench instantiates the `Pipeline_Top` module as the device under test. It also generates a VCD waveform file named `dump.vcd`, which can be viewed using a waveform viewer to observe signals such as the clock, reset, program counter, pipeline registers, ALU results, and forwarding signals during simulation.

```
1   module pipeline_tb();
2
3       reg clk=0, rst;
4
5       always begin
6           clk = ~clk;
7           #50;
8       end
9
10      initial begin
11          rst <= 1'b0;
12          #200;
13          rst <= 1'b1;
14          #1000;
15          $finish;
16      end
17
18      initial begin
19          $dumpfile("dump.vcd");
```

```
        $dumpvars(0);
    end

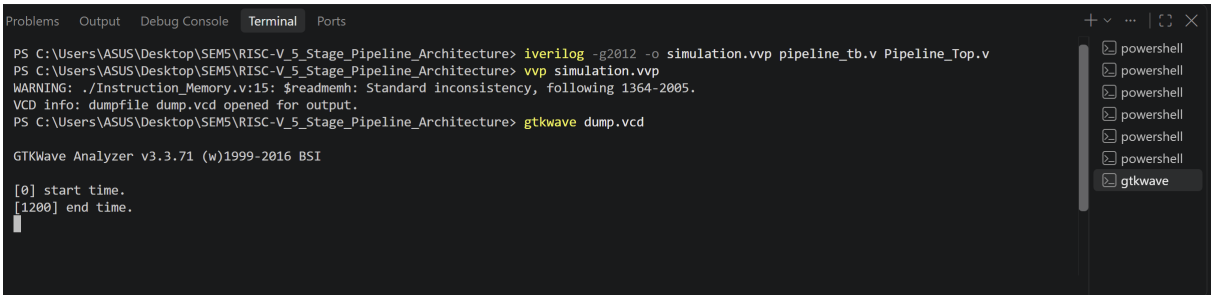
    Pipeline_Top dut (.clk(clk), .rst(rst));
endmodule
```

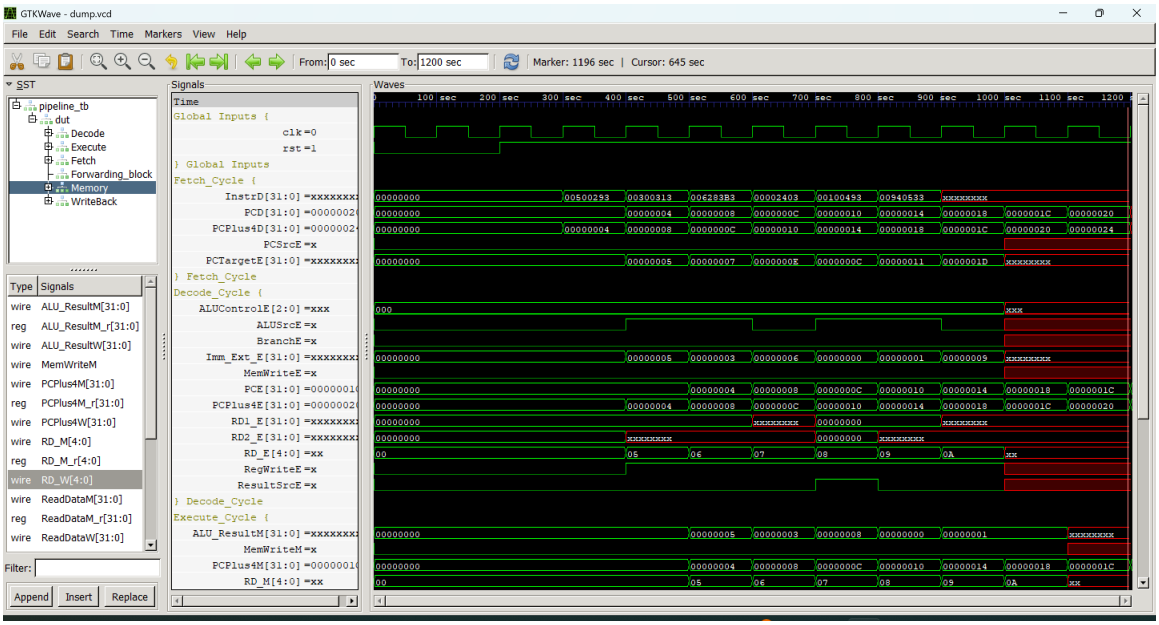
1.10 Instruction Sequence and Pipeline Operation

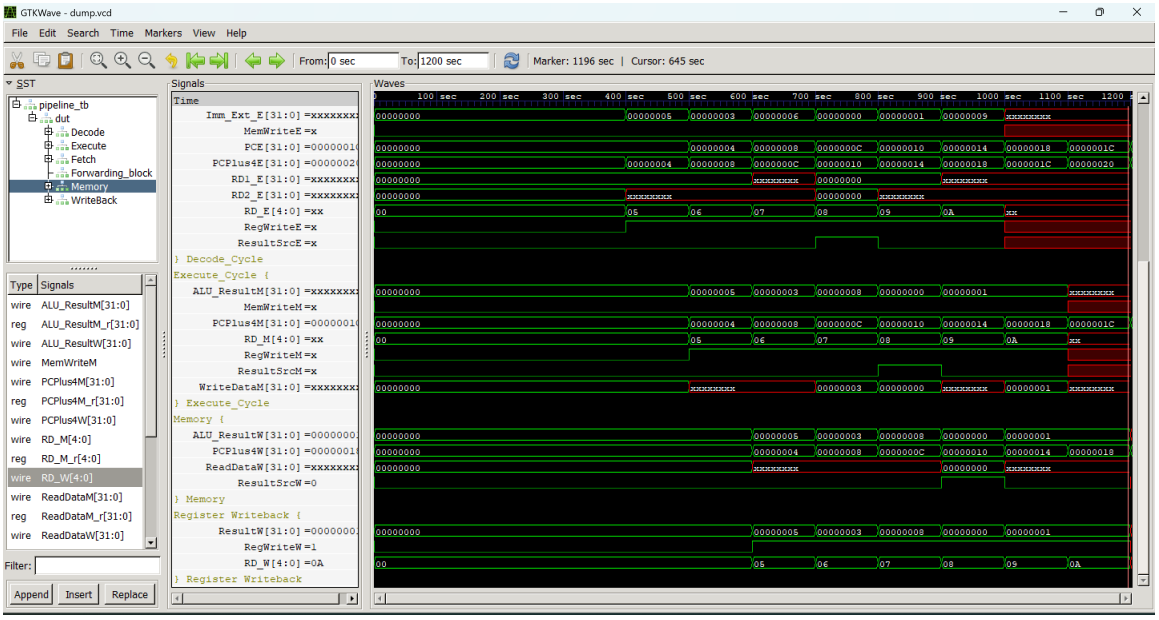
The instruction memory contains six instructions. The first two instructions initialize registers x5 and x6 with the values 5 and 3. The third instruction performs an addition using these two registers and stores the result in x7. The fourth instruction loads a value from memory into x8. The fifth instruction initializes x9 with the value 1, and the final instruction adds x8 and x9 and stores the result in x10.

The sequence is useful for demonstrating pipelined execution because instructions overlap across different stages. For example, while one instruction is being executed, another instruction can be decoded and a third instruction can be fetched. The dependencies between instructions also allow the forwarding logic to be exercised during simulation.

2 Simulation Waveform




GTKWave - dump.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 1200 sec
Marker: 1196 sec | Cursor: 645 sec
SST
pipeline_tb
dut
Decode
Execute
Fetch
Forwarding_block
Memory
WriteBack
Type Signals
wire ALU_ResultM[31:0]
reg ALU_ResultM_r[31:0]
wire ALU_ResultW[31:0]
wire MemWriteM
wire PCPlus4M[31:0]
reg PCPlus4M_r[31:0]
wire PCPlus4W[31:0]
wire RD_M[4:0]
reg RD_M_r[4:0]
wire RD_W[4:0]
wire ReadDataM[31:0]
reg ReadDataM_r[31:0]
wire ReadDataW[31:0]
Filter:
Append
Insert
Replace
Signals
Time
Global Inputs {
clk =0
rst =1
} Global Inputs
Fetch_Cycle {
InstrD[31:0] =xxxxxxxx
PCD[31:0] =00000020
PCPlus4D[31:0] =00000024
PCSrcE =x
PCTargetE[31:0] =xxxxxxxx
} Fetch_Cycle
Decode_Cycle {
ALUControlE[2:0] =xxx
ALUSrcE =x
BranchE =x
Imm_Ext_E[31:0] =xxxxxxxx
MemWriteE =x
PCE[31:0] =00000018
PCPlus4E[31:0] =00000020
RD1_E[31:0] =xxxxxxxx
RD2_E[31:0] =xxxxxxxx
RD_E[4:0] =xx
RegWriteE =x
ResultSrcE =x
} Decode_Cycle
Execute_Cycle {
ALU_ResultM[31:0] =xxxxxxxx
MemWriteM =x
PCPlus4M[31:0] =00000018
RD_M[4:0] =xx
Waves


GTKWave - dump.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 1200 sec
Marker: 1196 sec | Cursor: 645 sec
SST
pipeline_tb
dut
Decode
Execute
Fetch
Forwarding_block
Memory
WriteBack
Type Signals
wire ALU_ResultM[31:0]
reg ALU_ResultM_r[31:0]
wire ALU_ResultW[31:0]
wire MemWriteM
wire PCPlus4M[31:0]
reg PCPlus4M_r[31:0]
wire PCPlus4W[31:0]
wire RD_M[4:0]
reg RD_M_r[4:0]
wire RD_W[4:0]
wire ReadDataM[31:0]
reg ReadDataM_r[31:0]
wire ReadDataW[31:0]
Filter:
Append
Insert
Replace
Signals
Time
Imm_Ext_E[31:0] =xxxxxxxx
MemWriteE =x
PCE[31:0] =00000018
PCPlus4E[31:0] =00000020
RD1_E[31:0] =xxxxxxxx
RD2_E[31:0] =xxxxxxxx
RD_E[4:0] =xx
RegWriteE =x
ResultSrcE =x
} Decode_Cycle
Execute_Cycle {
ALU_ResultM[31:0] =xxxxxxxx
MemWriteM =x
PCPlus4M[31:0] =00000018
RD_M[4:0] =xx
RegWriteM =x
ResultSrcM =x
WriteDataM[31:0] =xxxxxxxx
} Execute_Cycle
Memory {
ALU_ResultW[31:0] =00000000
PCPlus4W[31:0] =00000018
ReadDataW[31:0] =xxxxxxxx
ResultSrcW =0
} Memory
Register Writeback {
ResultW[31:0] =00000000
RegWriteW =1
RD_W[4:0] =0A
} Register Writeback
Waves

3 Conclusion

The RISC-V processor was successfully implemented and verified using a five-stage pipelined architecture consisting of the Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. The waveform obtained from GTKWave clearly shows the parallel operation of these stages, where different instructions are present in different pipeline stages at the same time. The PC increases sequentially by 4 for each instruction, and the corresponding instruction codes can be observed progressing through the pipeline registers.

The instruction flow can be observed clearly from the waveform. Initially, the instructions `ADDI x5, x0, 5` and `ADDI x6, x0, 3` are fetched and decoded. When `ADD x7, x5, x6` reaches the Execute stage, it requires the values produced by the previous two instructions. This creates a data dependency. The Hazard Unit detects these dependencies by comparing the source registers with the destination registers in later pipeline stages and generates the forwarding control signals. The required values are therefore forwarded directly to the Execute stage instead of waiting for normal register writeback.

The next instruction, `LW x8, 0(x0)`, passes through the Execute and Memory stages to obtain the required data from memory. Meanwhile, `ADDI x9, x0, 1` continues through the pipeline. The final instruction, `ADD x10, x8, x9`, depends on both `x8` and `x9`. The waveform shows these instructions progressing through the pipeline while the required data is transferred between stages through the pipeline registers and forwarding paths. Thus, the simulation demonstrates how data dependencies are handled without stopping the normal flow of independent instructions.

Overall, the GTKWave results verify the operation of the five-stage pipeline and show the movement of instructions, PC values, ALU results, destination registers, memory data, and writeback results across the different stages. The implementation demonstrates the basic working of a pipelined RISC-V processor and shows how forwarding can be used to reduce the effect of data hazards while maintaining instruction overlap and improving the utilization of the processor datapath.